

Домашнее задание №8 по ОС.

Магомедов Абдул Омаргаджиевич. БПИ 234

Условие:

До 8 баллов

Модифицировать последнюю рассмотренную программу таким образом, чтобы данные в буфер записывались по кольцу и читались из него в том же порядке. Синхронизацию читателя и писателя при этом можно не менять. Сохранить также возможность работы одного писателя и одного читателя.

Опционально +1 балл

Использовать в программе с кольцевым буфером два отдельных мьютекса, **реализуемых посредством POSIX-семафоров**. Отдельно для читателей и писателей (аналогично решению в программе по курсу "Архитектура ВС", реализованному для многопоточного приложения).

Опционально +1 балл

Переработать код так, чтобы он допускал подключение до двух читателей при одном писателе с возможным завершением (как и реализовано в примере) всех процессов по завершению одного из них.

Предоставить только один вариант программ, учитывающих все реализованные ситуации. Опциональные решения можно реализовать как одну из альтернатив, если решение не планируется на 10 баллов. То есть каждая из них не зависит от другой. Дополнительно к отчету, описывающему проделанную работу, предоставить исходные тексты программ, решающих поставленную задачу на искомую оценку.

Оценка на 8 баллов:

Буфер реализован как **кольцевой массив** размером `BUF_SIZE = 10`.

Добавлены два новых поля в структуру `shared_memory`:

- `head` — индекс, в который писатель записывает данные.
- `tail` — индекс, из которого читатель считывает данные.

После записи или чтения индексы сдвигаются по кругу: $\text{index} = (\text{index} + 1) \% \text{BUF_SIZE}$.

Один писатель и один читатель взаимодействуют с буфером через разделяемую память.

Семафоры `empty`, `full`, `mutex` обеспечивают синхронизацию между ними (классическая схема).

Итог:

При запуске писатель последовательно заполняет ячейки буфера.

Читатель читает из буфера значения **в том же порядке**, в каком они были записаны.

Буфер "перезапускается" по кругу, когда достигает конца.

Оценка на 9 баллов:

Добавлены **два отдельных POSIX-мьютекса**:

- `mutex_writer` — защищает писателя (вместо общего `mutex`).
- `mutex_reader` — защищает критическую секцию читателя.

Использованы именованные семафоры:

- `/writer-mutex`
- `/reader-mutex`

Мьютексы инициализируются в `common.c`, закрываются и удаляются корректно.

В `writer.c` и `reader.c` изменена логика:

- Писатель блокирует `mutex_writer` только на момент записи.
- Читатель блокирует `mutex_reader` только на момент чтения.

Итог:

Улучшена параллельность и изоляция: писатель и читатели не конфликтуют, если работают с разными секциями буфера.

Позволяет масштабировать архитектуру при необходимости.

Оценка на 10 баллов:

В `shared_memory` добавлен счетчик `have_reader` вместо флага.

Изменена логика доступа читателей:

- Допускается максимум **два читателя**.
- Если третий читатель пытается подключиться — программа завершает его с сообщением.

Все процессы могут завершаться через `SIGINT`:

- При нажатии `Ctrl+C` в любом процессе, он отправляет сигнал остальным (`kill()` по `PID`).
- Очистка: все семафоры и `shm` удаляются при завершении.

Читатели считывают данные независимо, каждый сдвигает `tail`.

Итог:

В системе может одновременно работать:

- 1 писатель
- до 2-х читателей

При этом все процессы корректно завершаются при закрытии одного из них (обработка SIGINT/SIGTERM).